

Department of Computer Science & Engineering

TERM PAPER · THEMATIC ASSESSMENT, PHASE 2
Database Management Systems Laboratory

Persistent Memory Architecture in Agents Using DBMS

Comparison, Findings and Recommendation

Submitted by	Harshit Khemani	241302081
	Kush Ahuja	241302122
	Madhav Bassi	241302100
	Kushagra Agrawal	241302109
Programme	B.Tech, Computer Science & Engineering (AI/ML)	
Section	C	
Department	Computer Science & Engineering	
Institution	SGT University	
Date of submission	10 September 2026	

Abstract

Phase 1 of this term paper argued that the memory of an AI agent is a database. Phase 2 tests that argument against the systems and the benchmarks the field has actually published.

Four agent memory systems are read as data models: **MemGPT**, which pages a memory hierarchy the way an operating system pages virtual memory [4]; **Zep**, which stores memory as edges of a temporal knowledge graph and invalidates an edge instead of deleting it [1, 7]; **Mem0**, which gives a writer four explicit operations over a memory store [2]; and **A-MEM**, which argues that a fixed schema is itself the problem [6]. Two benchmarks supply the evidence about what fails: LongMemEval isolates five abilities of a long-term memory [3], and LoCoMo builds conversations of about 300 turns across as many as 35 sessions [5].

The comparison finds that three of the four have converged, from outside the database literature, on one shape: a subject–predicate–object triple carrying a validity interval, an entity registry that gives a thing one name, and a provenance edge back to the text that supported the claim. That is a database schema, and the questions the database community asked about it are fifty years old.

The findings are that structure pays and is measured to pay — Mem0 reports a 26% relative improvement over a deployed baseline and more than 90% fewer tokens than full context [2], and Zep reports up to 18.5% higher accuracy with 90% lower latency [1] — but that three guarantees a database declares are still missing everywhere. No system declares uniqueness, so identity is decided by similarity rather than by key. No paper reports what happens when two writers touch one memory at once. And no system uses the temporal standard that has existed since SQL:2011, although two of them implement its idea by hand.

The recommendation follows from those gaps: declare the schema where the data is, give the fact relation a temporal key, and make the update an operation the writer names rather than a side effect of a write.

KEYWORDS

agent memory, data model, bitemporal data, validity interval, entity resolution, lost update, SQL:2011, benchmark comparability.

WHAT THIS REPORT DOES NOT REPEAT

This is a standalone Phase 2 report. No section, table, figure or argument from the Phase 1 report appears here, and this phase does not carry over any earlier write-up of its own. It covers only the three criteria set for it: **comparison**, **findings** and **recommendation**. Every quantitative claim below is a figure a published, peer-reviewable paper reports about itself, cited at the point of use, so a reader can check each one at its source.

New Research Added in This Phase

The brief for this phase asks for three more research papers. The three below are new to this report, and each does real work in the sections that follow rather than sitting in the list.

Table 1. The three research papers added in Phase 2, and the work each one does.

Paper	What it contributes here	Used in
Zep , a temporal knowledge graph for agent memory [1, 7]	Zep attaches a validity interval to every edge and invalidates an old edge instead of deleting it, which is the bitemporal idea arriving in agent memory. It also supplies the only head-to-head accuracy figure in this report.	§1, §2.2, §2.5
Mem0 , scalable long-term memory with explicit operations [2]	Mem0 gives memory a small data manipulation language: the extractor decides ADD, UPDATE, DELETE or NOOP for every candidate. That names the axis Phase 1 left open, which is who owns the update.	§1, §2.2, §2.4
LongMemEval , a benchmark for long-term interactive memory [3]	It isolates five abilities of a long-term memory, two of which — temporal reasoning and knowledge updates — are exactly the two a validity interval exists to serve, and measures about a 30% accuracy drop for commercial assistants.	§1, §2.3

Five further sources are also new to this report. MemGPT [4] and LoCoMo [5] complete the comparison and the benchmark evidence; the SQL:2011 temporal standard [8], Gray’s account of the lost update [9] and Swoosh on entity resolution [10] supply the database results that the findings measure the systems against. A-MEM [6] states the opposing position on whether an agent memory should have a schema at all, and a recent survey [11] supplies the vocabulary that maps memory operations onto database operations.

Contents

Abstract	ii
New Research Added in This Phase	iii
1 Comparison	1
1.1 The four systems, and why these four	1
1.2 The same four operations, under two vocabularies	1
1.3 What the comparison separates	2
2 Findings	3
2.1 Three of the four converged on one schema	3
2.2 Structure is measured to pay, and each paper measures it differently	3
2.3 The measured failures are the ones a validity interval addresses . . .	4
2.4 The systems disagree about who owns the update	4
2.5 Identity is decided by similarity, not by key	5
2.6 Nobody reports what two writers do to one memory	5
2.7 The temporal standard already exists and none of them uses it	6
2.8 The results cannot be compared with each other	6
3 Recommendation	7
References	8

List of Figures

1 Three results, and when agent memory reached them	6
---	---

List of Tables

1 The three new research papers	iii
2 Four memory designs	2
3 Two vocabularies	2
4 Which parts each system holds	3
5 What each paper reports	4

1 Comparison

Phase 1 asked how an agent memory carries a fact across a session boundary. That separates designs but does not price them. This phase asks a narrower question with a harder answer. *Read four published systems as data models: what has each one declared, and what can each one refuse?*

1.1 The four systems, and why these four

The four are chosen because each takes a different position on the one question this topic turns on, which is how much structure the memory should declare in advance.

MemGPT takes the operating-system position. It treats the context window as physical memory and everything else as disk, and it moves data between the two tiers, “inspired by hierarchical memory systems in traditional operating systems that provide the appearance of large memory resources through data movement between fast and slow memory” [4]. The model itself issues the paging calls. There is no schema, because paging does not need one.

Zep takes the temporal-database position. Memory is a graph of entity nodes joined by edges, and every edge carries both when the fact was true and when the graph came to believe it, so a contradicted edge is invalidated rather than removed [1, 7].

Mem0 takes the data-manipulation position. Extraction proposes candidate memories, and the update phase resolves each candidate into one of four operations — ADD, UPDATE, DELETE or NOOP — against what is already stored [2].

A-MEM takes the opposing position, and the comparison is weaker without it. It argues that existing systems “rely on fixed schemas or operation structures” and that this limits adaptation to new tasks, proposing instead notes that link themselves into a network which emerges from content rather than from a declared shape [6].

1.2 The same four operations, under two vocabularies

The surveys of this field decompose a memory into six operations, and every one of the six already has a database name [11]. Table 3 is that translation, and it is the reason a database course has anything to say here at all.

Table 2. Four agent memory designs, read as data models. The last row is the one §2 measures.

	MemGPT [4]	Zep [1]	Mem0 [2]	A-MEM [6]
Unit of memory	A block of context, paged in and out	An edge between two entity nodes	A natural-language memory item	An atomic note that links itself
Schema	None. A memory hierarchy, not a relation	Declared: nodes, edges, intervals	Light, plus a graph variant	Rejected by design
Temporal model	None	Bitemporal: true-when and believed-when	Timestamps, no validity interval	None declared
Who owns the update	The model, by issuing a paging call	The engine, by invalidating the old edge	The writer, by naming one of four operations	The network, by re-linking notes
Identity of a thing	Whatever is in context	An entity node	By similarity	Emergent from links
Can the store refuse a bad write?	No	For what the engine declares	Only as far as the writer decides	No, by design

Table 3. What the agent-memory literature calls an operation, and what the database literature has called the same operation for decades.

In agent memory	In a database
Consolidation	Materialising a derived relation
Updating	The update anomaly, and the operation that avoids it
Indexing	Indexing
Forgetting	A retention policy
Retrieval	Query processing
Compression	A summary held as a materialised view

1.3 What the comparison separates

It does not separate them on ambition: all four are trying to make an agent remember across sessions, and all four report progress at it. It separates them on *what the store is allowed to promise*.

Read down the last row of Table 2. Only one of the four can refuse a write, and only for the part it has declared. In every other column the guarantee is a convention held by the program that writes, which means it holds exactly as long as every writer is well behaved. That is the axis §2 measures, and it is the axis on which fifty years of database results have something to say.

2 Findings

Most of the findings below carry the name of an older database phenomenon, because that is what they turn out to be.

2.1 Three of the four converged on one schema

F1 Zep, Mem0’s graph variant and the wider family all arrive at the same three parts, independently and from outside the database literature: a triple of subject, predicate and object; an interval saying when that triple was true; and an edge back to the text that supported it. Zep states the temporal half explicitly, giving each edge both the interval over which the fact held and the interval over which the graph believed it [1].

That is a schema, and naming it as one is the whole contribution of reading these systems as data models. A relation with a compound key, a validity interval and a provenance foreign key is not a new object. It is the object Codd’s successors spent the 1990s formalising, and the questions that follow — what is the key, what refuses a duplicate, what happens when two intervals overlap — are already answered.

The convergence is partial, and Table 4 is careful about where. Only Zep holds all four parts; Mem0 reaches the first and the last; and the two systems that declare no schema hold none of them, which is consistent rather than careless, because neither is trying to.

Table 4. Which parts of that schema each system holds. A dash means the system does not have the part, not that it fails at it: MemGPT and A-MEM are not trying to declare a schema.

Part of the schema	MemGPT	Zep	Mem0	A-MEM
A triple joining two entities	—	yes	graph variant	—
When the fact was true	—	yes	—	—
When the store believed it	—	yes	—	—
One canonical name for a thing	—	entity node	by similarity	emergent

2.2 Structure is measured to pay, and each paper measures it differently

F2 The case for structuring memory is not rhetorical. Both structured systems report gains against real baselines, and Table 5 collects what each one claims.

Table 5. What each paper reports about itself. The right-hand column is the finding: no two rows share a benchmark and a baseline, so these numbers rank nothing against each other.

System	Reported result	Measured on
Mem0 [2]	26% relative improvement in the LLM-as-a-judge metric over a deployed memory baseline; 91% lower p95 latency and over 90% token saving against full context	LOCOMO
Zep [1]	94.8% against 93.4% for MemGPT; and separately, up to 18.5% higher accuracy with 90% lower latency against baseline implementations	DMR, then LongMemEval

Two things follow, and they pull in opposite directions. The first is that the structured designs are not paying for their structure in speed: both report large latency and token reductions, because a memory that retrieves a few relevant rows sends less to the model than one that resends the conversation. The second is §2.8.

2.3 The measured failures are the ones a validity interval addresses

F3 LongMemEval isolates five abilities of a long-term memory, two of which — temporal reasoning and knowledge updates — are exactly the two a validity interval exists to serve, and it measures about a 30% accuracy drop for commercial assistants across sustained interactions [3].

LoCoMo reaches the same place from a different direction. Its conversations run to roughly 300 turns and as many as 35 sessions, and it finds that models “exhibit challenges in understanding lengthy conversations and comprehending long-range temporal and causal dynamics within dialogues”, still lagging human performance substantially even when given long context or retrieval [5].

Neither benchmark was built to argue for a schema. Both nevertheless locate the difficulty in the part of memory that changes over time, which is the part a temporal relation is for.

2.4 The systems disagree about who owns the update

F4 This is the update anomaly, and the four systems answer it in four different places. MemGPT gives the decision to the model, which issues the call that moves data between tiers [4]. Zep gives it to the engine, which closes the old edge when a new one contradicts it [1]. Mem0 gives it to the writer, which must resolve every candidate into ADD, UPDATE, DELETE or NOOP [2]. A-MEM gives it to the network, which re-links notes as they arrive [6].

Only Mem0's answer is a data manipulation language in the ordinary sense: a closed set of operations, each with a defined effect, chosen at the point of writing. The other three infer the operation from context, which is exactly the design a declared UPDATE was invented to replace.

2.5 Identity is decided by similarity, not by key

F5 A memory has to decide when two mentions are the same thing. Zep resolves this with an entity node, Mem0 by similarity, A-MEM by emergent linking, MemGPT not at all. None of the four declares uniqueness, and a store with no unique index cannot refuse the second introduction of a thing it already holds.

This has a theory, and the theory is older than the problem. Swoosh formalises match and merge as generic operators and defines the transitive closure a resolution has to reach before it is finished [10]: if *A* merges with *B* and *B* with *C*, the result is not settled until *A*, *B* and *C* are one. A pairwise similarity test does not reach that closure by itself, so duplicate identities survive in exactly the systems that resolve identity by similarity.

2.6 Nobody reports what two writers do to one memory

F6 Every one of the four papers evaluates a single writer. None of them reports a concurrent-write experiment, and none declares an isolation level.

This matters because agent memory is now shared by construction: a team of agents, or a team of people and their agents, writes to one store. The failure that follows is the oldest one in the transaction literature. Gray's account sets out the lost update and the degrees of consistency that permit it [9]: two writers read the same value, each computes a new one from what it read, and the second write erases the first.

Every design in Table 2 has that shape, because every one of them updates by reading first. Mem0 must retrieve the existing memories before it can decide whether a candidate is an ADD or an UPDATE [2]; Zep must find the edge a new fact contradicts before it can invalidate it [1]; A-MEM's evolution step revises the notes a new note links to, so its exposure grows with the linking [6]. In each case the decision is computed from a value read a moment earlier, and nothing in the write says what that value was.

The database answer is one line: make the write conditional on the state the reader saw, against a key the store has declared. That is what an ON CONFLICT clause or a compare-and-set against a version does, and it turns a lost update into a refused write the caller can retry. None of the four is documented as having one, and

until a paper measures two writers on one memory, nobody knows the size of the problem.

2.7 The temporal standard already exists and none of them uses it

F7 Zep implements two time axes by hand [1]. SQL:2011 standardised the same separation more than a decade earlier, as application-time period tables for when a row is true in the world and system-versioned tables for when the database held it, with period-scoped UPDATE and DELETE and automatic history rows [8].

The gap is not that the agent-memory systems are wrong. It is that they are re-deriving a solved problem, and paying for it in the usual way: a hand-built temporal column has no period-scoped update, no automatic history, and no way for the engine to refuse two intervals that overlap when they should not. Figure 1 puts that gap, and the two from §2.5 and §2.6, on one axis.

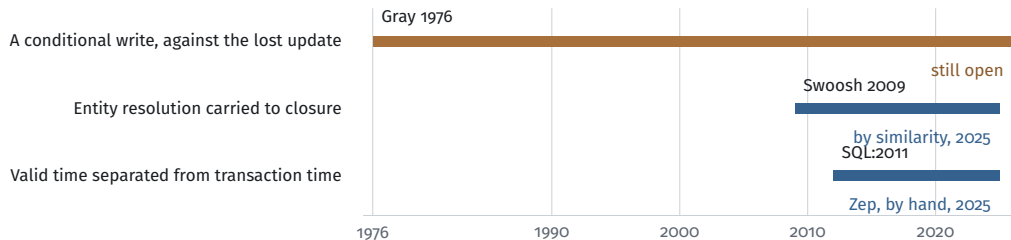


Figure 1. Three database results, and when the agent-memory literature reached them. Each bar runs from the year the database community settled the question to the year a system in this comparison acted on it. The clay bar is still open: no paper in §1 reports a concurrent-writer result at all.

2.8 The results cannot be compared with each other

F8 This finding cuts against the argument the rest of this report is making, and it belongs here for that reason.

Read Table 5 again as a comparison and it collapses. Mem0’s 26% is a relative improvement in an LLM-as-a-judge metric over a deployed baseline on LOCOMO [2]. Zep’s 94.8% is an absolute accuracy against MemGPT’s 93.4% on DMR, and its 18.5% is an improvement against baseline implementations on LongMemEval [1]. Different benchmarks, different baselines, different metrics, and in one case a relative figure set beside an absolute one.

So no ranking of these systems can be read off their own papers, and this report does not offer one. What the numbers jointly support is the weaker, sufficient claim of §2.2: structuring memory measurably beats resending the conversation. Anything stronger would need one benchmark, one baseline and one metric across all four, which nobody has yet run.

3 Recommendation

The recommendation is not that these systems should have been built on PostgreSQL. Three of the four earn their design: paging is the right answer to a context limit, an emergent network is a real position on adaptability, and both structured systems measurably beat resending the conversation. The recommendation is narrower. **Declare the parts that are already a schema, and let the store enforce them.**

Six rules follow. Each names the finding that forces it; F1 forces none, being the framing result rather than a defect.

1. **Declare the key where the data is.** A unique index on the canonical name of a thing, plus one ON CONFLICT statement, removes the window a read-then-write update cannot close and stops one real thing being introduced twice. (F5, F6)
2. **Give the fact relation a temporal key.** An exclusion constraint over a range type refuses the write that would let one subject hold two objects at once, so no scan has to find it afterwards. (F7)
3. **Use the standard, not a hand-built pair of columns.** Application-time period tables and system-versioned tables give period-scoped updates and automatic history that hand-rolled date columns do not [8]. (F7)
4. **Make the update an operation the writer names.** Mem0's ADD, UPDATE, DELETE and NOOP is a data manipulation language; inferring the operation from context is the anomaly the language replaced [2]. (F4)
5. **Resolve identity to closure, not pairwise.** Match and merge have to reach a transitive closure before a resolution is finished [10], and a similarity threshold does not reach it alone. (F5)
6. **Report the benchmark, the baseline and the metric together.** A number without all three cannot be compared with another number, which is the state of the field today. (F2, F8)

When, and with what caveat. These rules earn their cost once a memory is shared — more than one writer, or a store outliving the person who filled it. Below that line a paged context or an emergent network is the better engineering choice, and the structure is a liability. The caveat is F8: every quantitative claim here is a figure a system reports about itself on a benchmark of its own choosing, so this report compares designs and does not rank them. What it settles is firmer. A memory that arrives at triples with validity intervals, an entity registry and a provenance edge has arrived at a database schema, and the only question left is which of the guarantees it does without, and what it does instead.

References

Entries [1]–[3] are the three research papers added in this phase. Entries [4]–[7] complete the comparison and the benchmark evidence, and [8]–[11] are the database and survey results the findings measure against. Every entry is new to this report.

- [1] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan and D. Chalef. “Zep: A Temporal Knowledge Graph Architecture for Agent Memory.” arXiv:2501.13956, 2025.
<https://arxiv.org/abs/2501.13956>
- [2] P. Chhikara, D. Khant, S. Aryan, T. Singh and D. Yadav. “Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory.” *ECAI 2025*, Frontiers in Artificial Intelligence and Applications, vol. 413, pp. 2993–3000, 2025; arXiv:2504.19413. <https://arxiv.org/abs/2504.19413>
- [3] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang and D. Yu. “LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory.” *ICLR 2025*; arXiv:2410.10813. <https://arxiv.org/abs/2410.10813>
- [4] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders and J. E. Gonzalez. “MemGPT: Towards LLMs as Operating Systems.” arXiv:2310.08560, 2023.
<https://arxiv.org/abs/2310.08560>
- [5] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri and Y. Fang. “Evaluating Very Long-Term Conversational Memory of LLM Agents.” *ACL 2024*; arXiv:2402.17753. <https://arxiv.org/abs/2402.17753>
- [6] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan and Y. Zhang. “A-MEM: Agentic Memory for LLM Agents.” arXiv:2502.12110, 2025. <https://arxiv.org/abs/2502.12110>
- [7] Zep AI. “Graphiti — build temporal context graphs for AI agents.” GitHub, Apache-2.0 licence. <https://github.com/getzep/graphiti>
- [8] K. Kulkarni and J.-E. Michels. “Temporal Features in SQL:2011.” *ACM SIGMOD Record*, 41(3):34–43, 2012.
- [9] J. N. Gray, R. A. Lorie, G. R. Putzolu and I. L. Traiger. “Granularity of Locks and Degrees of Consistency in a Shared Data Base.” In *Modelling in Data Base Management Systems*, North Holland, 1976, pp. 365–394.
- [10] O. Benjelloun, H. Garcia-Molina, D. Menestrina, Q. Su, S. E. Whang and J. Widom. “Swoosh: A Generic Approach to Entity Resolution.” *The VLDB Journal*, 18(1):255–276, 2009.
- [11] Y. Du, W. Huang, D. Zheng, Z. Wang, S. Montella, M. Lapata, K.-F. Wong and J. Z. Pan. “Rethinking Memory in AI: Taxonomy, Operations, Topics, and Future Directions.” arXiv:2505.00675, 2025. <https://arxiv.org/abs/2505.00675>

B.Tech CSE (AI/ML), Section C · Department of Computer Science & Engineering, SGT University

Phase 2 · submitted 10 September 2026